DEVELOPER HANDOFF & SYSTEM TECHNICAL ARCHITECTURE GUIDE

Project: SRGH IVF Cryo Bank Storage & Record System
Author & Technical Lead: Aditya Kumar
Codebase Structure: Monorepo (`apps/frontend` , `apps/backend`)
Language/Runtime: TypeScript (Node.js v18+ / v20+)

---

1. Tech Stack Overview

Frontend Architecture (`apps/frontend`)

- **Framework**: React 18 + Vite
- **Styling**: Vanilla CSS + TailwindCSS (v3)
- **Icons**: Lucide React
- **State & API**: Custom fetch client with automatic token refreshing and caching (`apps/frontend/src/api/client.ts`)
- **PWA Support**: Web Manifest + iOS Safari / Android Chrome PWA installation components (`InstallAppModal.tsx`)

Backend Architecture (`apps/backend`)

- **Framework**: NestJS / Express REST API
- **ORM / Database**: Prisma ORM + PostgreSQL
- **Authentication**: Dual-layer security (Access Gate hash validation + JWT Access/Refresh tokens hashed with Argon2)
- **OCR Engine**: Google Cloud Vision API + Gemini AI dual extraction pipeline (`apps/backend/src/modules/ocr/`)
- **Background Tasks & Queue**: Redis + BullMQ (asynchronous PDF generation, email dispatches, and OCR processing)
- **Email Dispatch**: Nodemailer + SMTP (Gmail API integration)
- **PDF Generation**: PDFKit stream renderer for instant downloading & emailing of Cryo Storage Certificates and Thaw Reports

---

2. Codebase Directory Structure

```
ivf/
├── apps/
│   ├── frontend/
│   │   ├── src/
│   │   │   ├── api/
│   │   │   │   └── client.ts           # Centralized API fetch wrapper with token
handling
│   │   │   ├── assets/                 # Brand logos and images
│   │   │   ├── components/
│   │   │   │   ├── AccessGate.tsx      # Layer 1 Site Security Gate
│   │   │   │   ├── LoginForm.tsx       # Layer 2 Staff Login Form
│   │   │   │   ├── Dashboard.tsx       # Real-time storage heatmap & metrics
overview
```

```
|   |   |   |   ├── PatientForm.tsx      # Comprehensive multi-step patient
vitrification form
|   |   |   |   ├── PatientDirectory.tsx # Search, filter, and edit patient cryo
records
|   |   |   |   ├── ContainerView.tsx    # 3D interactive storage matrix (Cans 01-16 &
VisoTubes)
|   |   |   |   ├── OcrVerification.tsx  # Dual-engine OCR scanned record review &
confirmation
|   |   |   |   ├── ThawWorkflow.tsx     # Thaw authorization, straw removal, &
certificate generation
|   |   |   |   ├── InstallAppModal.tsx  # Mobile PWA installation instructions modal
|   |   |   |   ├── AdminPanel.tsx       # Staff user management & system settings
|   |   |   |   ├── AuditLogs.tsx        # Immutable audit log viewer
|   |   |   |   └── DeveloperSupport.tsx # System diagnostic & support tools
|   |   |   ├── context/
|   |   |   |   └── BackgroundTaskContext.tsx # Async queue status polling
|   |   |   ├── utils/
|   |   |   |   └── heatmap.ts           # 8-step storage utilization color
calculation
|   |   |   ├── App.tsx                  # Main app navigation & routing
|   |   |   └── main.tsx
|   |   ├── public/
|   |   |   └── manifest.json            # PWA app installation manifest
|   |   ├── package.json
|   |   └── vite.config.ts
|   |
|   └── backend/
|       ├── prisma/
|       |   ├── schema.prisma            # PostgreSQL Database Schema definition
|       |   └── seed.ts                  # Database seeding script (Demo patients &
default accounts)
|       ├── src/
|       |   ├── common/
|       |   |   ├── config.ts            # Environment variables loader & validator
|       |   |   └── prisma.ts            # Prisma Client singleton instance
|       |   ├── modules/
|       |   |   ├── auth/                # Auth controller & service (Login, Refresh,
Access Key)
|       |   |   ├── patient/             # Patient CRUD & Vitrification record
management
|       |   |   ├── storage/             # Can, VisoTube, & Straw capacity allocation
logic
|       |   |   ├── ocr/                 # Google Cloud Vision & Gemini AI OCR
processing
|       |   |   ├── thaw/                # Thaw workflow & certificate generation
|       |   |   ├── audit/               # Immutable audit logging service
|       |   |   └── mail/                # SMTP Email notification service
|       |   └── main.ts                  # NestJS server bootstrapping & port listener
|       └── package.json
|
├── CLIENT_HANDOFF_DOCS/                 # Complete client & developer handoff
documentation package
```

```
├── docs/                                # Technical specification markdown files
├── production_schema.sql                # Complete PostgreSQL production SQL schema
backup
├── .env                                 # Master environment configuration
└── package.json                         # Root monorepo workspace configuration
```

---

3. Local Development Setup Guide

Prerequisites

- Node.js v18.x or v20.x
- npm v9+
- PostgreSQL database server running locally on port 5432
- Redis server running on port 6379 (optional for background email queues)

Installation Steps

1. **Clone & Install Dependencies**:

   ```
   git clone https://github.com/Aditya07024/IVF-Storage-Clinic-Record-System.git
   cd IVF-Storage-Clinic-Record-System
   npm install
   ```

2. **Configure Environment Variables**: Copy `.env.example` to `.env` in root and configure database connection:

   ```
   cp .env.example .env
   ```

3. **Initialize Database Schema & Seed Data**:

   ```
   cd apps/backend
   npx prisma db push
   npx prisma db seed
   cd ../..
   ```

4. **Start Development Servers**:

   ```
   # From project root:
   npm run dev
   ```

   - **Frontend**: Accessible at `http://localhost:3000`
   - **Backend API**: Accessible at `http://localhost:4000`

---

4. Production Deployment & Maintenance

PM2 Process Management Commands

The production app is managed using PM2 on Linux:

```
# Check running services
pm2 status

# Restart backend & frontend services
pm2 restart all

# View real-time server logs
pm2 logs backend
pm2 logs frontend

# Stop services
pm2 stop all
```

Nginx Reverse Proxy Configuration

Nginx handles SSL termination and proxies requests:

- `https://www.sgrhivfcryo.in/` -> Proxy to `http://127.0.0.1:3000`
- `https://api.sgrhivfcryo.in` -> Proxy to `http://127.0.0.1:4000`

To restart Nginx after domain configuration updates:

```
sudo systemctl restart nginx
```

Database Backup & Restore

1. Backup Production Database:

```
pg_dump -U ivf_admin -h 127.0.0.1 -d ivf_production > /backups/ivf_backup_$(date +%Y%m%d_%H%M%S).sql
```

2. Restore Database from SQL dump:

```
psql -U ivf_admin -h 127.0.0.1 -d ivf_production < /backups/ivf_backup_file.sql
```

---

5. Storage Hierarchy & Business Rules

1. **Can Hierarchy**:
    - Total Cans: **16 Cans** (Can 01 through Can 16).
    - Viso Tubes: **8 Viso Tubes** per Can (Viso Tube 01 to 08).
    - Straw Limit: **Maximum 2 embryos/oocytes per straw**.
2. **Access Key Hash**:
    - The Layer 1 Access Key hash is calculated using SHA-256 in config.ts.
    - Any raw input key whose SHA-256 digest matches `APP_ACCESS_KEY_HASH` is granted Layer 1 access.
3. **OCR Processing**:

- Uses `Google Cloud Vision API` for text detection and passes raw text to `Gemini 1.5 Flash` to extract structured JSON fields (`patientName` , `husbandName` , `vitrificationDate` , `embryoCount` , `strawId` , `canNumber` , `visoTube`).